Manual Técnico

Sistema Integral Web de Control de Producción y Distribución de Alimentos Congelados — Distribuidora Yaneth

Apéndice A — Arquitectura, instalación, configuración, respaldos y mantenimiento

Versión 1.0 — Octubre 2026

Elaborado por: Dennis Cornel

Contenido

1. Introducción y alcance 4
2. Arquitectura del sistema 4
2.1 Visión general 4
2.2 Recorrido de una petición 5
2.3 Tecnologías y versiones 6
2.4 Estructura del código 6
2.5 Seguridad y control de acceso 7
2.6 Documentación de la API 8
3. Requisitos 8
3.1 Hardware y software del servidor 8
3.2 Requisitos de los equipos cliente 8
3.3 Accesos y datos necesarios antes de instalar 8
4. Variables de entorno 9
4.1 Backend (archivo Backend/.env) 9
4.2 Frontend (archivo Frontend/.env.production) 9
5. Instalación en el servidor 10
5.1 Día 1 – Preparación del servidor 10
5.2 Día 2 – Migración y configuración 11
5.3 Día 3 – Configuración inicial de datos y pruebas 13
6. Operación de los servicios 13
6.1 Orden de arranque 13
6.2 Comandos de operación 14
6.3 Registros (logs) 14
7. Respaldos y restauración 14
7.1 Qué se respalda 14
7.2 Respaldo diario automático 14
7.3 Restauración 15
7.4 Prueba periódica de restauración 15
8. Mantenimiento 15
8.1 Rutinas de mantenimiento 15

8.2 Actualizar a una nueva versión del sistema 16
8.3 Migraciones de la base de datos 16
8.4 Gestión de usuarios y accesos 16
8.5 Parámetros operativos 16
8.6 Solución de problemas 17
8.7 Buenas prácticas de seguridad 17
Anexo A. Checklist de aceptación de la instalación 17

1. Introducción y alcance

Este manual describe cómo está construido el Sistema Integral Web de Control de Producción y Distribución de Alimentos Congelados, cómo se instala en el servidor de Distribuidora Yaneth y cómo se opera, respalda y mantiene una vez en producción. Está dirigido a la persona encargada del servidor y a cualquier desarrollador que dé continuidad al sistema.

Forma parte del paquete de apéndices que se entrega junto con el código fuente:

Apéndice	Contenido	Ubicación en el repositorio del backend
A	Manual técnico (este documento)	docs/A_Manual_Tecnico/
B	Manual de usuario, por rol y por módulo	docs/B_Manual_de_Usuario/
C	Esquema de la base de datos y diagrama entidad-relación	docs/C_Esquema_BD/ (remite a sql/ y diagramas/base_datos/)
D	Código fuente	Repositorios distri_yaneth_backend y distri_yaneth_frontend

El procedimiento de instalación (capítulo 5) corresponde al plan de tres jornadas resumido en el Capítulo V de la tesis; aquí se detallan los comandos de cada actividad.

2. Arquitectura del sistema

2.1 Visión general

El sistema es una aplicación web de tres capas con procesamiento asíncrono de las operaciones de base de datos. Se compone de cuatro piezas:

- **Cliente web (React + Vite).** Interfaz que usa el personal desde el navegador. Se compila a archivos estáticos (HTML, CSS y JavaScript) que sirve un servidor web (Nginx, Apache o IIS).
- **API REST (Node.js + Express 5).** Recibe las peticiones del cliente, valida el token de sesión (JWT) y el rol del usuario, y encola cada operación de base de datos.
- **Cola de trabajos (BullMQ sobre Redis).** Desacopla la petición HTTP de su procesamiento y limita cuántas operaciones se ejecutan en paralelo contra la base de datos.
- **Worker (Node.js).** Toma los trabajos de la cola y ejecuta las consultas y transacciones contra MySQL/MariaDB. Corre dentro del mismo proceso que la API: al iniciar la API se inicia también el worker.

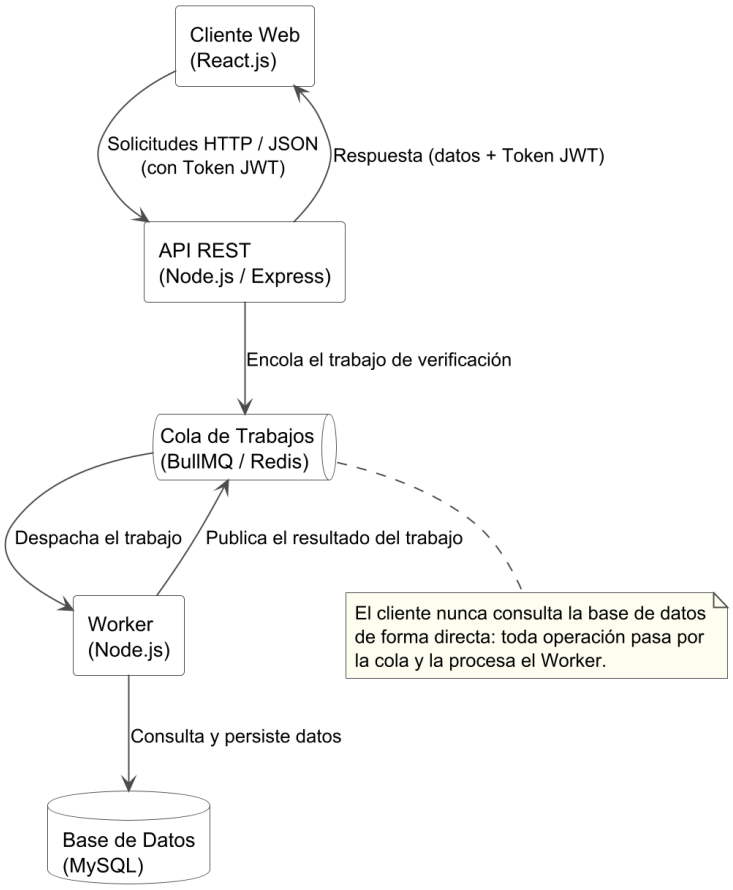


Figura 1. Arquitectura de componentes del sistema.

El cliente nunca consulta la base de datos de forma directa: toda operación pasa por la API, la cola y el worker. Redis solo guarda trabajos en tránsito (se eliminan 60 segundos después de completarse); toda la información del negocio vive en la base de datos.

2.2 Recorrido de una petición

1. El usuario inicia sesión en `POST /api/users/login`. La API valida la contraseña (bcrypt) y devuelve un token JWT con el identificador, nombre y rol del usuario, con vigencia definida por `JWT_EXPIRES_IN` (8 h por defecto).
2. El cliente envía el token en la cabecera `Authorization: Bearer <token>` en cada petición posterior.
3. El middleware `authenticateToken` verifica la firma y la vigencia del token; `authorizeRoles` comprueba que el rol tenga acceso a la ruta solicitada (respuestas 401 o 403 si no).
4. El controlador valida los datos de entrada y llama al modelo, que encola un trabajo en BullMQ (`enqueueDbJob`) y espera su resultado hasta 60 segundos.

5. El worker ejecuta el manejador del dominio correspondiente (`src/workers/domains/*.worker.js`). Las operaciones que modifican varias tablas se ejecutan en una transacción y registran el cambio en la tabla `auditoria_cambios`.
6. El resultado vuelve por la cola a la API, que responde al cliente en formato JSON.

2.3 Tecnologías y versiones

Tabla 1. Tecnologías del sistema (según package.json de cada repositorio).

Capa	Tecnología	Versión declarada
Backend	Node.js	20.19 o superior (recomendado 22 LTS)
Backend	Express	5.2
Backend	BullMQ / ioredis	5.76 / 5.10
Backend	mysql2 (pool de conexiones)	3.22
Backend	jsonwebtoken / bcryptjs	9.0 / 3.0
Backend	swagger-jsdoc / swagger-ui-express	6.3 / 5.0
Backend	morgan (registro de peticiones), cors, dotenv	1.10 / 2.8 / 17.4
Frontend	React / React DOM	19.2
Frontend	Vite (compilación)	8.0
Frontend	Recharts (gráficas del dashboard)	3.10
Frontend	jsPDF + jspdf-autotable / ExcelJS (exportaciones)	4.2 + 5.0 / 4.4
Frontend	SweetAlert2 (notificaciones)	11.26
Datos	MySQL 8 o MariaDB 10.4+	Motor InnoDB, juego de caracteres utf8mb4
Cola	Redis (o Memurai en Windows)	6.2 o superior

2.4 Estructura del código

El código está dividido en dos repositorios independientes:

Tabla 2. Estructura del repositorio del backend.

Carpeta (backend)	Contenido
src/server.js, src/app.js	Arranque: conexión a la base de datos, cola y worker; registro de rutas y permisos por rol.
src/config/	Pool de MySQL (db.js), conexión a Redis (redis.js) y especificación Swagger (swagger.js).
src/middlewares/	auth.middleware.js: validación del token JWT y autorización por rol.
src/routes/	Definición de endpoints por módulo, con su documentación Swagger.
src/controllers/	Validación de entrada y armado de la respuesta HTTP.
src/models/	Encolan los trabajos de base de datos de cada módulo.
src/queues/	db.queue.js: cola BullMQ y espera de resultados.
src/workers/	db.worker.js (registro de manejadores), domains/ (lógica y SQL por módulo) y shared/ (constantes, auditoría, códigos de fase).
sql/schema.sql	Esquema consolidado de la base de datos (34 tablas).
sql/migrations/	Cambios incrementales fechados, para actualizar bases ya existentes.
diagramas/	Fuentes PlantUML y PNG de los diagramas (incluye el diagrama entidad-relación).
scripts/	Utilidades de desarrollo (siembra de datos de demostración). No se

	ejecutan en producción.
docs/	Apéndices A, B y C.

Tabla 3. Estructura del repositorio del frontend.

Carpeta (frontend)	Contenido
src/pages/	Página de inicio de sesión y contenedor principal (Dashboard).
src/components/	Menú lateral, widgets y gráficas del dashboard, componentes comunes.
src/features/	Un módulo por carpeta (entries, maturation, production, orders, routes, returns, traceability, etc.).
src/services/	Llamadas HTTP a la API; todas usan la variable VITE_API_URL.
src/config/rolePermissions.js	Módulos visibles por rol y agrupación del menú lateral.
src/utils/	Formato de códigos de fase, notificaciones y utilidades de exportación.

2.5 Seguridad y control de acceso

- Contraseñas almacenadas con hash bcrypt (10 rondas).
- Sesión sin estado mediante JWT firmado con `JWT_SECRET`; al vencer, el usuario debe iniciar sesión de nuevo.
- Cuatro roles: Administrador, Producción, Logística y Piloto. El menú del cliente muestra solo los módulos del rol, y la API vuelve a validar el rol en cada ruta (la restricción no depende solo de la interfaz).
- Borrado lógico: los registros se marcan como Inactivos en lugar de eliminarse, lo que preserva la trazabilidad.
- Auditoría: cada alta, modificación o baja queda en `auditoria_cambios` con el usuario, la fecha y los valores antes y después del cambio.

Tabla 4. Permisos por prefijo de la API (src/app.js).

Prefijo de la API	Roles con acceso
/api/users (gestión de usuarios)	Administrador (el inicio de sesión es público)
/api/proveedores, /api/productos, /api/inventario, /api/entradas-mercancia, /api/maduracion, /api/redes-verdes, /api/trazabilidad	Administrador, Producción, Logística
/api/produccion	Administrador, Producción
/api/clientes, /api/tipos-servicio, /api/servicios-vehiculo	Administrador, Logística
/api/pedidos, /api/rutas, /api/reporte-kilometraje-vehiculo	Administrador, Logística, Piloto
/api/devoluciones	Administrador, Logística, Piloto, Producción (recepción: Administrador y Logística; resolución: además Producción)
/api/vehiculos	Consulta: Administrador, Logística, Piloto; altas y cambios: Administrador, Logística
/api/auditoria	Administrador

2.6 Documentación de la API

La API publica su documentación interactiva (OpenAPI/Swagger) y un punto de verificación de estado:

Ruta	Uso
GET /health	Verifica que la API esté en línea. Responde {"ok": true, "message": "API funcionando"}.
GET /api-docs	Documentación interactiva de todos los endpoints (Swagger UI).
GET /api-docs.json	Especificación OpenAPI en formato JSON.

3. Requisitos

3.1 Hardware y software del servidor

Tabla 5. Requisitos del servidor.

Componente	Requisito mínimo	Requisito recomendado
Servidor de aplicación	2 vCPU, 4 GB RAM, 40 GB SSD	4 vCPU, 8 GB RAM, 80 GB SSD
Sistema operativo	Ubuntu Server 22.04 LTS o Windows Server 2019+	Ubuntu Server 22.04/24.04 LTS
Node.js	20.19	22 LTS
Base de datos	MySQL 8.x o MariaDB 10.4+	Instancia dedicada con respaldos automáticos
Redis	6.2, 1 GB RAM	7.x, 2 GB RAM con persistencia AOF (Memurai en Windows)
Servidor web	Nginx, Apache o IIS	Nginx con certificado HTTPS
Red	Enlace estable con IP fija o DNS dinámico	Enlace redundante o plan de datos de respaldo

3.2 Requisitos de los equipos cliente

- Navegador actualizado: Google Chrome, Microsoft Edge o Mozilla Firefox.
- Pantalla de al menos 1366 × 768 en oficina y planta. Los pilotos pueden usar un teléfono o tableta: la interfaz se adapta al ancho de pantalla.
- Acceso de red al dominio o IP del servidor.

3.3 Accesos y datos necesarios antes de instalar

- Acceso de administrador (root/sudo o Administrador de Windows) al servidor.
- Credenciales para crear la base de datos y el usuario de aplicación en MySQL/MariaDB.
- Dominio o IP fija con la que el personal accederá al sistema.
- Lista real del personal (nombre completo, usuario y rol) para el alta de usuarios.

- Catálogos base de la empresa: clientes, proveedores, vehículos, productos por tipo, tipos de etapa (Pelado, Corte, Fritura y Embalaje), tipos de merma, cuartos fríos y el umbral de Grados Brix vigente.

4. Variables de entorno

4.1 Backend (archivo Backend/.env)

Se crean copiando la plantilla `.env.example`. Los valores de la plantilla son de ejemplo y nunca se usan en producción.

Tabla 6. Variables de entorno del backend.

Variable	Descripción	Valor por defecto
PORT	Puerto en el que escucha la API.	3000
DB_HOST / DB_PORT	Servidor y puerto de MySQL/MariaDB.	localhost / 3306
DB_USER / DB_PASSWORD	Usuario de aplicación (no usar root).	root / (vacío)
DB_NAME	Nombre de la base de datos.	proyecto_graduacion
REDIS_HOST / REDIS_PORT	Servidor y puerto de Redis.	127.0.0.1 / 6379
REDIS_USERNAME / REDIS_PASSWORD	Credenciales de Redis, si las tiene configuradas.	(vacío)
REDIS_DB	Número de base lógica de Redis usada por la cola.	0
DB_QUEUE_NAME	Nombre de la cola BullMQ. Debe ser distinto si dos instancias comparten Redis.	db-queries
DB_QUEUE_CONCURRENCY	Trabajos que el worker procesa en paralelo.	5
MATURATION_BRIX_THRESHOLD	Grados Brix a partir de los cuales un control de maduración pasa el sub-lote a "Listo para producción".	22
JWT_SECRET	Secreto de firma de los tokens. Generar uno largo y aleatorio y mantenerlo privado.	dev-secret-change-me (inseguro)
JWT_EXPIRES_IN	Vigencia de la sesión.	8h
DB_*_TABLE	Nombres de tablas (p. ej. DB_USERS_TABLE). Existen por compatibilidad; no deben modificarse.	Nombres del esquema

Para generar un secreto seguro:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

4.2 Frontend (archivo Frontend/.env.production)

Variable	Descripción	Ejemplo
VITE_API_URL	URL base de la API. Se incrusta en los archivos compilados: si cambia, hay que volver a ejecutar npm run build.	https://sistema.yaneth.com (con proxy /api) o http://IP-servidor:3000

Nota: El cliente arma las rutas como VITE_API_URL + "/api/…". Si el servidor web publica la API en el mismo dominio mediante un proxy de /api (sección 5.2, actividad 2.7), VITE_API_URL es simplemente el dominio del sistema.

5. Instalación en el servidor

La instalación limpia se organiza en tres jornadas de aproximadamente seis horas, con márgenes para imprevistos. Los comandos corresponden a Ubuntu Server; en Windows se indican las alternativas.

Día	Enfoque	Duración
Día 1	Preparación del servidor	6 h
Día 2	Migración y configuración	6 h
Día 3	Configuración inicial de datos y pruebas	6 h

5.1 Día 1 – Preparación del servidor

1.1 Actualizar el sistema operativo (30 min)

```
sudo apt update && sudo apt upgrade -y
```

1.2 Instalar Node.js 22 LTS (20 min)

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
node -v && npm -v
```

1.3 Instalar MySQL y crear la base de datos (45 min)

Se crea un usuario de aplicación con permisos solo sobre la base del sistema; el backend no usa la cuenta root.

```
sudo apt install -y mysql-server
sudo mysql_secure_installation

sudo mysql -u root -p
CREATE DATABASE proyecto_graduacion CHARACTER SET utf8mb4 COLLATE
utf8mb4_general_ci;
CREATE USER 'app_yaneth'@'localhost' IDENTIFIED BY '<contraseña-segura>';
GRANT ALL PRIVILEGES ON proyecto_graduacion.* TO 'app_yaneth'@'localhost';
FLUSH PRIVILEGES;
```

1.4 Instalar Redis (45 min)

En Windows, al no existir una versión oficial de Redis, se usa Memurai (compatible con el protocolo) o Redis en un contenedor Docker.

```
sudo apt install -y redis-server
sudo systemctl enable redis-server --now
redis-cli ping   # debe responder PONG
```

1.5 Configurar el firewall (30 min)

Solo se exponen SSH y el puerto web. MySQL (3306), Redis (6379) y la API (3000) quedan accesibles únicamente desde el propio servidor.

```
sudo ufw allow 22/tcp
sudo ufw allow 80,443/tcp
sudo ufw enable
sudo ufw status
```

1.6 Copiar el código fuente (30 min)

```
sudo mkdir -p /opt/yaneth-sistema && cd /opt/yaneth-sistema
git clone https://github.com/DecosUniversity/distri_yaneth_backend.git Backend
git clone https://github.com/DecosUniversity/distri_yaneth_frontend.git Frontend
```

1.7 Programar los respaldos automáticos (45 min)

Ver el procedimiento completo en el capítulo 7.

1.8 Documentar la configuración y probar conectividad (60 min)

Registrar credenciales, rutas y versiones instaladas en un documento interno, y verificar que el servidor responda desde la red de la empresa. Margen para imprevistos: 40 min.

5.2 Día 2 – Migración y configuración

2.1 Crear el esquema de la base de datos (30 min)

```
cd /opt/yaneth-sistema/Backend
mysql -u app_yaneth -p proyecto_graduacion < sql/schema.sql
```

Importante: schema.sql elimina y vuelve a crear cada tabla (DROP TABLE IF EXISTS). Se ejecuta una sola vez sobre la base vacía; nunca sobre una base con datos. Para actualizar una base existente se usan las migraciones (sección 8.3).

2.2 Verificar las tablas (30 min)

```
SELECT COUNT(*) FROM information_schema.tables
WHERE table_schema = 'proyecto_graduacion';   -- debe devolver 34
```

2.3 Crear el usuario administrador inicial (15 min)

El esquema no incluye usuarios. Se genera el hash bcrypt de una contraseña temporal y se inserta el primer Administrador; los demás usuarios se crean desde el módulo Usuarios.

```
cd /opt/yaneth-sistema/Backend && npm install --omit=dev
node -e "console.log(require('bcryptjs').hashSync('<contraseña-temporal>', 10))"

INSERT INTO usuarios (nombre_completo, username, password_hash, rol)
VALUES ('Administrador del sistema', 'admin', '<hash-generado>', 'Administrador');
```

2.4 Configurar las variables de entorno (30 min)

```
cp .env.example .env
nano .env
# ajustar: DB_USER, DB_PASSWORD, DB_NAME, REDIS_*, JWT_SECRET, JWT_EXPIRES_IN
```

2.5 Publicar la API como servicio administrado (45 min)

Se usa PM2 para que la API (con su worker) se reinicie automáticamente ante una falla o un reinicio del servidor. En Windows puede usarse NSSM para registrar `node src/server.js` como servicio.

```
sudo npm install -g pm2
cd /opt/yaneth-sistema/Backend
pm2 start src/server.js --name yaneth-api
pm2 save
pm2 startup   # ejecutar el comando que indique
```

2.6 Compilar el frontend (30 min)

```
cd /opt/yaneth-sistema/Frontend
echo "VITE_API_URL=https://<dominio-del-sistema>" > .env.production
npm ci
npm run build
sudo mkdir -p /var/www/yaneth && sudo cp -r dist/* /var/www/yaneth/
```

2.7 Configurar el servidor web (30 min)

Nginx sirve los archivos del cliente y reenvía /api a la API local. Después se activa HTTPS con Certbot.

```
server {
    listen 80;
    server_name <dominio-del-sistema>;
    root /var/www/yaneth;
    index index.html;

    location /api/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location / {
        try_files $uri /index.html;
    }
}

sudo nginx -t && sudo systemctl reload nginx
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d <dominio-del-sistema>
```

2.8 Prueba de conexión extremo a extremo (45 min)

```
curl http://localhost:3000/health        # {"ok":true,"message":"API funcionando"}
pm2 logs yaneth-api --lines 20          # "Conexion a MySQL establecida" y
"Servidor escuchando en puerto 3000"
```

Después, abrir el dominio en un navegador e iniciar sesión con el administrador inicial.

2.9 Explicación del entorno técnico al responsable del servidor (40 min)

Recorrido por la arquitectura, ubicación de los registros (logs) y reinicio de servicios (capítulo 6). Ajustes y margen para imprevistos: 55 min.

5.3 Día 3 – Configuración inicial de datos y pruebas

Tabla 7. Actividades del Día 3.

Actividad	Detalle	Tiempo
3.1 Cambiar la contraseña del administrador	Desde Usuarios → Contraseña, reemplazando la temporal.	10 min
3.2 Cargar catálogos maestros	Clientes, proveedores, vehículos, tipos de servicio y productos por tipo (Materia Prima, Producto Terminado, Insumo y Venta Directa). Cada Producto Terminado se crea con su plan de etapas.	45 min
3.3 Catálogos de producción	Tipos de etapa (Pelado, Corte, Fritura, Embalaje), tipos de merma y cuartos fríos. Se crean desde los formularios de Producción y Maduración ("+ Nuevo tipo…").	20 min
3.4 Estándar de maduración	Confirmar el umbral de Grados Brix (MATURATION_BRIX_THRESHOLD) y reiniciar la API si se cambió.	10 min
3.5 Alta de usuarios reales	Una cuenta por persona con su rol: Administrador, Producción, Logística o Piloto.	30 min
3.6 Capacitación breve	Registro correcto de la información propia de cada rol (Manual de usuario, Apéndice B).	45 min
3.7 Prueba de aceptación por módulo	Con datos reales: entrada de materia prima, control de maduración, proceso de producción con etapas, pedido, entrega y un reporte.	45 min
3.8 Tiempos de respuesta	Consultas y reportes en menos de 3 segundos en uso normal.	20 min
3.9 Validar respaldos	Ejecutar el respaldo manualmente y restaurarlo en una base de prueba (capítulo 7).	20 min
3.10 Aceptación final	Revisión con el responsable de la empresa y firma del Anexo A.	45 min
3.11 Ajustes finales	Atención de observaciones. Margen para imprevistos: 25 min.	45 min

6. Operación de los servicios

6.1 Orden de arranque

La API verifica al iniciar la conexión a la base de datos y a Redis; si alguno no responde, el proceso termina con el mensaje "No se pudo iniciar el servidor". Por eso el orden de arranque es:

1. MySQL/MariaDB
2. Redis
3. API + worker (yaneth-api)
4. Servidor web (Nginx)

Con los servicios habilitados (systemctl enable y pm2 startup), este orden se respeta automáticamente al reiniciar el servidor.

6.2 Comandos de operación

Tabla 8. Comandos frecuentes de operación (Linux con PM2).

Acción	Comando
Estado de la API	pm2 status
Reiniciar la API	pm2 restart yaneth-api
Detener / iniciar la API	pm2 stop yaneth-api / pm2 start yaneth-api
Ver registros de la API	pm2 logs yaneth-api --lines 100
Estado de MySQL / Redis / Nginx	sudo systemctl status mysql | redis-server | nginx
Reiniciar MySQL / Redis	sudo systemctl restart mysql | redis-server
Verificar la API	curl http://localhost:3000/health
Verificar Redis	redis-cli ping

En Windows, los servicios equivalentes se administran desde services.msc (MySQL/MariaDB, Memurai y el servicio NSSM de la API) o con `net stop` / `net start <servicio>`.

6.3 Registros (logs)

Componente	Dónde	Qué se registra
API	pm2 logs yaneth-api (~/.pm2/logs/)	Cada petición HTTP (morgan), errores 500 y arranque.
MySQL	/var/log/mysql/error.log	Arranque, apagado y errores del motor.
Redis	/var/log/redis/redis-server.log	Arranque y persistencia.
Nginx	/var/log/nginx/access.log y error.log	Accesos y errores del servidor web.
Negocio	Tabla auditoria_cambios	Quién cambió qué registro, cuándo y con qué valores.

Para evitar que los registros de PM2 llenen el disco se recomienda instalar la rotación:

```
pm2 install pm2-logrotate
```

7. Respaldos y restauración

7.1 Qué se respalda

- **Base de datos (obligatorio).** Contiene toda la información del negocio.
- **Archivo Backend/.env.** Contiene credenciales y el secreto JWT; se guarda aparte, en un lugar seguro.
- **Redis no requiere respaldo:** solo contiene trabajos en tránsito.
- **Código:** está en los repositorios Git; basta con conocer la versión (commit) instalada.

7.2 Respaldo diario automático

Se programa un volcado diario a las 02:00 con retención de siete días. El parámetro --single-transaction obtiene una copia consistente sin bloquear el uso del sistema.

```
sudo mkdir -p /respaldos && sudo chown $USER /respaldos
printf '[mysqldump]\nuser=app_yaneth\npassword=<contraseña>\n' > ~/.my.cnf &&
chmod 600 ~/.my.cnf

crontab -e
# respaldo diario 02:00 y limpieza de copias con más de 7 días
0 2 * * * mysqldump --single-transaction --routines proyecto_graduacion | gzip >
/respaldos/yaneth_$(date +\%F).sql.gz
30 2 * * * find /respaldos -name 'yaneth_*.sql.gz' -mtime +7 -delete
```

Al menos una vez por semana debe copiarse el respaldo más reciente fuera del servidor (disco externo o almacenamiento en la nube) para protegerlo ante una falla del equipo.

En Windows se programa la misma tarea con el Programador de tareas, ejecutando mysqldump.exe a diario.

7.3 Restauración

1. Detener la API para que nadie registre datos durante la restauración: `pm2 stop yaneth-api`.
2. Respaldar el estado actual, aunque esté dañado, por si se necesita revisarlo después.
3. Restaurar el archivo elegido:

```
gunzip -c /respaldos/yaneth_2026-10-21.sql.gz | mysql -u app_yaneth -p
proyecto_graduacion
```

1. Iniciar la API (`pm2 start yaneth-api`), comprobar /health e iniciar sesión.

7.4 Prueba periódica de restauración

Una vez al mes se restaura el último respaldo en una base de prueba (por ejemplo proyecto_graduacion_prueba) y se comprueba el número de tablas y algunos registros recientes. Un respaldo que no se ha probado no garantiza la recuperación.

8. Mantenimiento

8.1 Rutinas de mantenimiento

Tabla 9. Plan de mantenimiento preventivo.

Frecuencia	Tarea
Diaria	Verificar que exista el respaldo del día. Revisar que la API esté en línea (pm2 status).
Semanal	Copiar el último respaldo fuera del servidor. Revisar errores en pm2 logs. Revisar espacio en disco (df -h).
Mensual	Prueba de restauración. Actualizaciones de seguridad del sistema operativo (apt upgrade) en horario sin operación. Revisar usuarios activos y desactivar los del personal que ya no labora.
Trimestral	Revisar versiones de Node.js, MySQL y Redis; renovar certificados si no son automáticos.

8.2 Actualizar a una nueva versión del sistema

1. Avisar al personal y elegir un horario sin operación.
2. Hacer un respaldo manual de la base de datos (sección 7.2).
3. Descargar la nueva versión y revisar si incluye migraciones nuevas en sql/migrations/.
4. Aplicar las migraciones pendientes en orden de fecha (sección 8.3).
5. Actualizar dependencias y reiniciar la API.
6. Recompilar y publicar el frontend.
7. Verificar /health, iniciar sesión y recorrer los módulos principales.

```
cd /opt/yaneth-sistema/Backend && git pull && npm ci --omit=dev && pm2 restart
yaneth-api
cd /opt/yaneth-sistema/Frontend && git pull && npm ci && npm run build && sudo cp
-r dist/* /var/www/yaneth/
```

8.3 Migraciones de la base de datos

Cada cambio de estructura se guarda en sql/migrations/ con el formato AAAA-MM-DD_descripcion.sql. schema.sql ya contiene todos los cambios hasta su fecha de generación (9 de octubre de 2026); por ello, en una instalación nueva no se ejecutan migraciones. En una base existente se aplican, en orden, solo las migraciones posteriores a la última aplicada:

```
mysql -u app_yaneth -p proyecto_graduacion < sql/migrations/2026-10-
09_add_codigo_trazabilidad.sql
```

Se recomienda llevar una bitácora con la fecha de aplicación de cada migración.

8.4 Gestión de usuarios y accesos

- Altas, cambios de rol, restablecimiento de contraseña y bajas se hacen desde el módulo Usuarios (rol Administrador).
- Un usuario dado de baja queda Inactivo: no puede iniciar sesión, pero sus registros históricos se conservan.
- Si se pierde el acceso de todos los administradores, se genera un nuevo hash y se actualiza directamente en la base:

```
UPDATE usuarios SET password_hash = '<hash-bcrypt>' WHERE username = 'admin';
```

8.5 Parámetros operativos

Parámetro	Dónde se cambia
Umbral de Grados Brix para "Listo para producción"	MATURATION_BRIX_THRESHOLD en .env y reinicio de la API.
Duración de la sesión	JWT_EXPIRES_IN en .env y reinicio de la API.

Tipos de etapa, tipos de merma y cuartos fríos	Desde la interfaz, en los formularios de Producción y Maduración.
Plan de etapas de cada producto terminado	Producción → pestaña Etapas por producto.
Stock mínimo y precio sugerido de cada producto	Módulo Productos.

8.6 Solución de problemas

Tabla 10. Problemas frecuentes y su solución.

Síntoma	Causa probable	Acción
La API no inicia: "No se pudo iniciar el servidor"	MySQL o Redis detenidos, o credenciales incorrectas en .env.	Verificar systemctl status mysql y redis-cli ping; revisar DB_* y REDIS_*.
Las pantallas cargan pero las operaciones tardan ~60 s y fallan	Redis no responde o el worker se detuvo.	redis-cli ping; pm2 restart yaneth-api; revisar pm2 logs.
"Token invalido o expirado" / regreso al inicio de sesión	La sesión venció (JWT_EXPIRES_IN).	Iniciar sesión de nuevo.
Error 403 al usar un módulo	El rol del usuario no tiene acceso a esa ruta.	Revisar el rol en Usuarios.
El navegador no se conecta a la API	VITE_API_URL incorrecta, proxy de Nginx o firewall.	Revisar .env.production, recompilar el frontend y probar curl /health desde el servidor.
MySQL no arranca tras un corte de energía	Apagado brusco que dañó archivos del motor.	Revisar el log de errores de MySQL; si no se recupera, restaurar el último respaldo (capítulo 7). Se recomienda UPS para el servidor.
Disco lleno	Respaldos o registros acumulados.	Revisar /respaldos, la retención y pm2-logrotate.

Los mensajes de validación del negocio (por ejemplo "El peso no cuadra" o "Faltan etapas requeridas") no son fallas técnicas: se explican en el Manual de usuario.

8.7 Buenas prácticas de seguridad

- Usar un JWT_SECRET propio, largo y aleatorio; si se sospecha que se filtró, cambiarlo (todas las sesiones se cierran).
- Publicar el sistema solo por HTTPS.
- No exponer los puertos 3000, 3306 ni 6379 fuera del servidor.
- Contraseñas individuales para cada persona: no compartir cuentas.
- Mantener el archivo .env fuera de cualquier repositorio y con permisos restringidos (chmod 600).

Anexo A. Checklist de aceptación de la instalación

Marcar cada actividad al completarse y firmar al finalizar la instalación.

- ☐ Servidor aprovisionado y actualizado

- ☐ Node.js, MySQL/MariaDB y Redis instalados y en ejecución
- ☐ Firewall configurado; 3000, 3306 y 6379 sin acceso externo
- ☐ Esquema creado (34 tablas) y administrador inicial registrado
- ☐ Variables de entorno configuradas con credenciales reales y JWT_SECRET propio
- ☐ API corriendo como servicio con reinicio automático
- ☐ Frontend compilado y publicado con HTTPS
- ☐ Respaldo automático configurado y restauración probada
- ☐ Catálogos maestros y de producción cargados
- ☐ Usuarios reales dados de alta con su rol
- ☐ Prueba de aceptación por módulo sin observaciones abiertas
- ☐ Tiempos de respuesta verificados (< 3 s)

Firma de conformidad: _____ Fecha: _____